WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Boolean Functions

Lecture 04: truth tables, and the canonical sum-of-products and product-of-sums forms

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Boolean functions

02 Sum of products

03 Product of sums

04 Working with canonical forms

05 Summary

01

Boolean functions

What is a Boolean function?

Definition (Boolean function)

A *Boolean function* of n variables is a function

$$f : \{0,1\}^n \to \{0,1\}$$

built from the Boolean operations. Put simply: anything that takes one or more 0/1 inputs and gives one 0/1 output.

- $f(x, y, z) = xy + z$ is a Boolean function of three variables.
- So is any lookup table that lists every 3-bit input on the left and one bit on the right: the table defines the function.

The truth table

row	x	y	z	xy	$f = xy + z$
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	0
3	0	1	1	0	1
4	1	0	0	0	0
5	1	0	1	0	1
6	1	1	0	1	1
7	1	1	1	1	1

- One row for every input: $2^3 = 8$ rows.
- Rows are listed in counting order. The row number is the input xyz read as a binary number: row 6 is $x = 1$, $y = 1$, $z = 0$.
- To fill it in, evaluate the expression row by row; helper columns, like xy, keep it tidy.

What a truth table is, and is not

A truth table is

- **tabular**, not algebraic;
- **declarative**, not imperative: it says *which* output belongs to each input, not *how* to compute it;
- **exhaustive**: every input must be there. With 10 inputs that is $2^{10} = 1024$ rows.

To build a circuit we need an algebraic form that says how to compute f. The next two sections give two standard ones, the canonical forms, that can be read straight off any truth table.

How many Boolean functions are there?

A function of n variables is a truth table with 2^n rows, and each row's output is 0 or 1. So there are 2^{2^n} different functions.

n	rows	functions
1	2	4
2	4	16
3	8	256
4	16	65,536

The four functions of one variable

x	0	x'	x	1
0	0	1	0	1
1	0	0	1	1

Many of the 16 functions of two variables have names: AND, OR, XOR, NAND, NOR, … They become the logic gates of Unit 6.

02

Sum of products

Detecting one row

Build a function that is 1 for the input on one row, and 0 everywhere else. Take row 2: $x = 0$, $y = 1$, $z = 0$.

$$r_2(x, y, z) = \begin{cases} 1 & \text{if } x = 0 \text{ and } y = 1 \text{ and } z = 0, \\ 0 & \text{otherwise.} \end{cases}$$

- A product is 1 only when *every* factor is 1.
- x' is 1 exactly when $x = 0$; y when $y = 1$; z' when $z = 0$.
- So $r_2(x, y, z) = x'yz'$.

Each variable appears once: as it is where its bit is 1, complemented where it is 0.

Minterms

> **Definition (Minterm)**
>
> A *minterm* of n variables is a product in which every variable appears once, plain or complemented. The minterm m_i is 1 on row i only.

A row detector like r_2 is a minterm: $r_2 = m_2$.

row	x	y	z	minterm
0	0	0	0	$m_0 = x'y'z'$
1	0	0	1	$m_1 = x'y'z$
2	0	1	0	$m_2 = x'yz'$
3	0	1	1	$m_3 = x'yz$
4	1	0	0	$m_4 = xy'z'$
5	1	0	1	$m_5 = xy'z$
6	1	1	0	$m_6 = xyz'$
7	1	1	1	$m_7 = xyz$

The canonical sum of products

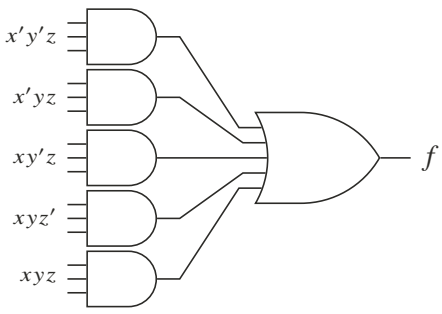
$f = xy + z$ is 1 on rows 1, 3, 5, 6 and 7. So f is 1 exactly when *one of* their detectors is 1: OR them together.

$$f = m_1 + m_3 + m_5 + m_6 + m_7 = x'y'z + x'yz + xy'z + xyz' + xyz$$

> **Sum of minterms**
>
> Every Boolean function is the sum of the minterms of the rows where it is 1. In short: $f(x, y, z) = \sum m(1, 3, 5, 6, 7)$.

For a given order of variables this form is unique: two expressions are the same function exactly when they have the same minterms.

As a circuit: AND–OR



- One AND gate per minterm: its output is 1 only when it sees its row.
- One OR gate collects them: f is 1 when any row detector is 1.
- Complemented inputs come from NOT gates (not drawn).

The book's Figure 2.1 shows the same circuit in Logisim-evolution. Gates in detail: Unit 6.

03

Product of sums

Detecting the zeros

Instead, detect the rows where $f = 0$: rows 0, 2 and 4.

- $m_0 + m_2 + m_4$ is 1 exactly where f is 0: it is f'.
- So f is its complement. Apply De Morgan (Lecture 03), twice:

$$
\begin{aligned}
f &= (m_0 + m_2 + m_4)' = m_0' \cdot m_2' \cdot m_4' && \text{De Morgan} \\
&= (x'y'z')' \cdot (x'yz')' \cdot (xy'z')' \\
&= (x + y + z)(x + y' + z)(x' + y + z) && \text{De Morgan, involution}
\end{aligned}
$$

Maxterms

Definition (Maxterm)

A *maxterm* of n variables is a sum in which every variable appears once, plain or complemented. The maxterm $M_i = m_i'$ is 0 on row i only.

Each variable appears as it is where its bit is 0, complemented where it is 1: the opposite of a minterm.

row	x	y	z	maxterm
0	0	0	0	$M_0 = x + y + z$
1	0	0	1	$M_1 = x + y + z'$
2	0	1	0	$M_2 = x + y' + z$
3	0	1	1	$M_3 = x + y' + z'$
4	1	0	0	$M_4 = x' + y + z$
5	1	0	1	$M_5 = x' + y + z'$
6	1	1	0	$M_6 = x' + y' + z$
7	1	1	1	$M_7 = x' + y' + z'$

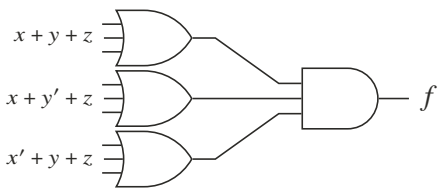
The canonical product of sums

Product of maxterms

Every Boolean function is the product of the maxterms of the rows where it is 0:

$$f(x, y, z) = \prod M(0, 2, 4).$$

- A product is 0 when any factor is 0, and M_i is 0 only on row i.
- So f is 0 on rows 0, 2 and 4, and 1 everywhere else.



OR–AND circuit; compare Figure 2.2 of the book.

Two forms, one function

$$f = \sum m(1, 3, 5, 6, 7) = \prod M(0, 2, 4)$$

- The two index lists together contain every row exactly once: the minterms are the 1s, the maxterms are the 0s.
- The complement swaps the roles: $f' = \sum m(0, 2, 4) = \prod M(1, 3, 5, 6, 7)$.
- The sum of products has one term per 1, the product of sums one term per 0. Use the form with fewer terms.

> **Common mistake**
>
> In a minterm a 0 bit gives a complemented variable; in a maxterm a 0 bit gives a plain one. Row 4 ($x = 1$, $y = 0$, $z = 0$): $m_4 = xy'z'$, but $M_4 = x' + y + z$.

04

Working with canonical forms

From an expression to the sum of products

Multiply each product by $(v + v') = 1$ for every variable v it is missing, expand, and drop repeated terms.

$$
\begin{aligned}
xy + z &= xy(z + z') + (x + x')(y + y')\,z && \texttt{complement, identity} \\
&= xyz + xyz' + x'y'z + x'yz + xy'z + xyz && \texttt{distributivity} \\
&= xyz + xyz' + x'y'z + x'yz + xy'z && \texttt{idempotence} \\
&= \textstyle\sum m(1, 3, 5, 6, 7)
\end{aligned}
$$

The same five minterms as the truth table gave us.

From an expression to the product of sums

First turn the expression into a product of sums; then add $vv' = 0$ for every missing variable v, and split with the second distributive law.

$$xy + z = (x + z)(y + z) \qquad \text{distributivity}$$
$$x + z = x + yy' + z = (x + y + z)(x + y' + z) \qquad \text{complement, distributivity}$$
$$y + z = xx' + y + z = (x + y + z)(x' + y + z) \qquad \text{complement, distributivity}$$

Multiply, and keep $(x + y + z)$ once (idempotence):

$$xy + z = (x + y + z)(x + y' + z)(x' + y + z) = \prod M(0, 2, 4).$$

Try it: the majority function

$maj(x, y, z)$ is 1 when at least two of its three inputs are 1.

x	y	z	maj
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

1. Fill in the truth table.
2. Write the canonical sum of products.
   $\sum m(3, 5, 6, 7) = x'yz + xy'z + xyz' + xyz$
3. Write the canonical product of sums.
   $\prod M(0, 1, 2, 4) = (x+y+z)(x+y+z')(x+y'+z)(x'+y+z)$

The majority of three bits is the carry when you add them (Unit 16).

05

Summary

Summary

- A Boolean function maps n input bits to one output bit. Its truth table has 2^n rows; there are 2^{2^n} such functions.
- A minterm m_i is 1 on row i only; a maxterm $M_i = m_i'$ is 0 on row i only.
- Canonical sum of products: the minterms of the 1-rows, $\sum m(\dots)$, built as AND–OR.
- Canonical product of sums: the maxterms of the 0-rows, $\prod M(\dots)$, built as OR–AND.
- Both forms are unique, but long. Next lecture: making them short.

Before the next lecture

- Read Section 2.3 of the book, up to the end of *Canonical Representations*.
- Write $\sum m(0, 3, 5, 6)$ as a product of maxterms, and write out both forms in full.
- Expand $x + y'z$ into its canonical sum of products. How many minterms does it have?

> **Next lecture** (Unit 5)
>
> Simplifying Boolean functions with Karnaugh maps.